

# Primer: Specifying Expressions for Event Binning

---

This primer explains how to define **numeric expressions** that select events for each “bin” (condition) in the `epoch_average_MEG_BIDS_phase8b.py` workflow.

Expressions operate directly on the **integer event codes** from `STI101` (or from annotation-timed events with those same codes).

Each expression returns `True` or `False` for each event, determining whether that event belongs in the bin.

---

## Concept

Every event carries a **16-bit integer** from the MEGIN trigger channel (`STI101`).

Users often encode information by **setting individual bits** or **groups of bits** (bit fields) to represent experimental factors such as:

| Bit(s) | Meaning                   | Example Value                        |
|--------|---------------------------|--------------------------------------|
| 1–3    | Face identity (0–7)       | <code>face_3</code> → bits 1–3 = 011 |
| 4–6    | Context / ISI category    | Short, Long, Variable                |
| 7      | Target flag               | 1 = target, 0 = non-target           |
| 11–14  | Block or run number (1–8) | Run 5 → bits 11–14 = 0101            |

Your **expression** is a logical statement evaluated on the variable `code`, the integer value for each event.

If the expression is `True`, that event is included in the bin.

---

## Syntax Rules

- Expressions are evaluated using Python syntax with **bitwise operators**:
    - `&` (AND)
    - `|` (OR)
    - `~` (NOT)
    - `^` (XOR)
    - Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
  - Combine logical clauses with bitwise `&` and `|` instead of `and` / `or`.
  - Parentheses `()` group sub-expressions.
  - Hex (`0x...`) and decimal numbers are both allowed.
  - You can use inline **helper functions** for readability:
    - `bit(n)` — True if bit *n* (1-based) is set.  
Example: `bit(7)` → True for target events.
    - `field(start, width)` — Extracts the integer value from a consecutive bitfield.  
Example: `field(1,3)` → bits 1–3 interpreted as an integer (face ID).
    - `anymask(mask)` — True if **any** bits in `mask` are set.
    - `allmask(mask)` — True if **all** bits in `mask` are set.
-

## Examples

### 1. Simple equality on a bit mask

Select events where bit 7 (target flag) is ON:

```
expr: "(code & 0x40) != 0"
```

Equivalent helper-based form:

```
expr: "bit(7)"
```

---

### 2. Select all faces that are **not** targets

Require that any of bits 4–6 are set (i.e., a valid face stimulus),  
and that bit 7 is **not** set:

```
expr: "((code & 0x38) != 0) & ((code & 0x40) == 0)"
```

or equivalently:

```
expr: "anymask(0x38) & ~bit(7)"
```

---

### 3. Select a specific face identity

(face 3, bits 1–3 = 011)

```
expr: "field(1,3) == 3"
```

---

### 4. Combine multiple factors

(Target faces in **block 1**)

```
expr: "bit(7) & (field(11,4) == 1)"
```

---

### 5. Exclude certain bits

Faces with a context code in bits 4–6 but **not** variable ISI (bit 6):

```
expr: "anymask(0x38) & ~bit(6)"
```

---

## 6. Match several codes explicitly

Include only codes 1280 or 2304 (decimal):

```
expr: "(code == 1280) | (code == 2304)"
```

---

## 7. Check multiple bits must be set

Bits 1 and 3 both ON:

```
expr: "bit(1) & bit(3)"
```

Equivalent mask form:

```
expr: "allmask((1<<0) | (1<<2))"
```

---

## 8. Numeric ranges on bit-fields

Run number (bits 11–14) between 2 and 5 inclusive:

```
expr: "(field(11,4) >= 2) & (field(11,4) <= 5)"
```

---

## Tips for Users

### 1. Preview counts

Run the pipeline with `--quiet` off: it prints how many events each expression selects.

### 2. Use hex for clarity

Bit masks are easier to read as hex (`0x40` = bit 7, `0x38` = bits 4–6).

### 3. Avoid `and` / `or`

Use `&` and `|` — they operate element-wise across all events.

### 4. Test incrementally

Start with simple conditions (e.g., one bit) before combining clauses.

## 5. Document meanings

Add a short comment in YAML next to each `expr` explaining what bits it refers to.

---

### Example YAML snippet

```
conditions:
  faces_non_target:
    expr: "anymask(0x38) & ~bit(7)"      # any face, not target
  faces_target:
    expr: "bit(7)"                      # all targets
  face3_block1:
    expr: "(field(1,3) == 3) & (field(11,4) == 1)"
```

This structure lets collaborators define bins succinctly and reproducibly—no need to enumerate dozens of raw trigger codes.